

UNITAT 4 · SETMANES 5-6

Processos i memòria

📅 Setmanes 5-6 · dates per confirmar </> Laboratori TD3 (resolt + repte IA) ~ 7 animacions

LABORATORI TD3 · RESULT

Memòries intermèdies, `fork`, `exec` i `wait`

Què fa la biblioteca de C per sobre de les crides al sistema, què es copia quan un procés es duplica, i com un programa en llança un altre i s'assabenta de com ha acabat.

EXERCICIS

- 1 Crida al sistema o funció de biblioteca?
- 2 Els tres modes de memòria intermèdia de stdio
- 3 `fork` i la memòria intermèdia duplicada
- 4 `launch` : redirigir i executar un altre programa
- 5 Exploració: la memòria virtual i la vida d'un procés (nou)

TD3 · 1

● Result

Crida al sistema o funció de biblioteca?

Enunciat. Classifica: `printf`, `fopen`, `fclose`, `fread`, `open`, `close`, `lseek`, `rewind`, `write`, `exit`, `_exit`. Fes servir els seus prototips i la seva secció del manual.

Solució

CRIDES AL SISTEMA (SECCIÓ 2)

`open`, `close`, `lseek`, `write`,
`_exit`

FUNCIONS DE LA BIBLIOTECA DE C (SECCIÓ 3)

`printf`, `fopen`, `fclose`, `fread`, `rewind`,
`exit`

La pista és als tipus: les crides treballen amb descriptors (`int fd`); les funcions de stdio, amb `FILE *`, una estructura de la biblioteca que conté un descriptor i una

memòria intermèdia. `exit` buida aquestes memòries intermèdies i executa les funcions `atexit` abans de cridar `_exit`, que acaba el procés de seguida.

TD3 · 2

● Result · sortides reals

Els tres modes de memòria intermèdia de stdio

Enunciat. `buffer.c` escriu «Hello », dorm un segon i escriu «World!». Amb macros del preprocessador tria, per a cada cadena, `fprintf` o `write`, `stdout` o `stderr`, i al final `exit` o `_exit`. Executa cada versió al terminal i a través d'una canonada (`2>&1 | cat -u`) i identifica els tres modes de memòria intermèdia.

Solució

C tds/td3/buffer.c

```
/*
 * TD3 - exercise 2: the three buffering modes of the C library.
 * One source, several behaviours chosen at compile time (see the Makefile
 * variants):
 *   STRING1, STRING2           the two strings (default "Hello " and
 *                               "World!\n")
 *   USE_FPRINTF1/2             write string 1/2 with fprintf (buffered) instead
 *                               of write (not buffered)
 *   USE_STDERR1/2              write string 1/2 on stderr instead of stdout
 *   USE__EXIT                   end with _exit() (no buffer flush) instead of
 *                               exit()
 * Run each variant twice: ./buffer_N (terminal) and ./buffer_N 2>&1 |
 * cat -u (pipe)
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#include "utils.h"

#ifndef STRING1
#define STRING1 "Hello "
#endif
#ifndef STRING2
```

```

#define STRING2 "World!\n"
#endif

#ifdef USE_STDERR1
#define STREAM1 stderr
#define FD1 STDERR_FILENO
#else
#define STREAM1 stdout
#define FD1 STDOUT_FILENO
#endif

#ifdef USE_STDERR2
#define STREAM2 stderr
#define FD2 STDERR_FILENO
#else
#define STREAM2 stdout
#define FD2 STDOUT_FILENO
#endif

int main(void)
{
#ifdef USE_FPRINTF1
    fputs(STRING1, STREAM1);           /* goes to the
    stdio buffer */
#else
    exit_if(write(FD1, STRING1, strlen(STRING1)) == -1, "write"); /*
    straight to the kernel */
#endif

    sleep(1);

#ifdef USE_FPRINTF2
    fputs(STRING2, STREAM2);
#else
    exit_if(write(FD2, STRING2, strlen(STRING2)) == -1, "write");
#endif

#ifdef USE__EXIT
    _exit(EXIT_SUCCESS);               /* system call: the stdio buffers are NOT
    flushed */
#else
    exit(EXIT_SUCCESS);                /* library function: flushes the stdio
    buffers, then _exit */
#endif
}

```

VERSIÓ (MAKEFILE)	TERMINAL	CANONADA	PER QUÈ
<pre>buffer_1: fprintf(stdout) + write</pre>	<pre>World! · Hello</pre>	<pre>World! · Hello</pre>	«Hello » no té <code>\n</code> : ni tan sols la memòria intermèdia per línia el buida; només surt en l' <code>exit</code> .
<pre>buffer_2: write + fprintf(stderr)</pre>	<pre>Hello World!</pre>	<pre>Hello World!</pre>	<code>stderr</code> no té memòria intermèdia: cada <code>fprintf</code> és un <code>write</code> immediat.
<pre>buffer_3: fprintf(stdout) + fprintf(stderr)</pre>	<pre>World! · Hello</pre>	<pre>World! · Hello</pre>	El flux d'error surt de seguida; la sortida estàndard espera.
<pre>buffer_4: com 3, amb "Hello\n"</pre>	<pre>Hello · World!</pre>	<pre>World! · Hello</pre>	En un terminal <code>stdout</code> té memòria intermèdia <i>per línia</i> (el <code>\n</code> la buida); en una canonada és <i>completa</i> (4 KiB), així que espera l' <code>exit</code> .
<pre>buffer_5: fprintf(stdout) + _exit</pre>	<pre>World!</pre>	<pre>World!</pre>	<code>_exit</code> no buida la memòria intermèdia: «Hello » es perd.

Sortida real

```
$ ./buffer_4          # en una terminal
Hello
World!
```

Sortida real

```
$ ./buffer_4 2>&1 | cat -u
World!
Hello
```

Sortida real

```
$ ./buffer_5 2>&1 | cat -u
World!
```

Els tres modes: **sense memòria intermèdia** (`stderr`), **per línia** (`stdout` cap a un terminal) i **completa** (`stdout` cap a un fitxer o una canonada). `write` mai no té una memòria intermèdia visible per al programa: el que lliura, el nucli ja ho té.

Animació A4.6 · Memòries intermèdies de stdio →

TD3 · 3

● **Result · provat**

fork i la memòria intermèdia duplicada

Enunciat. Un programa imprimeix el seu pid **sense salt de línia** amb `printf` i es duplica amb `fork`; el fill imprimeix el seu pid i el del pare. El pare no ha d'imprimir res després del `fork`. Què passa? Proposa tres solucions.

Solució

```
C tds/td3/fork_pid.c

/*
 * TD3 - exercise 3: print the pid WITHOUT a newline, fork, and let only the
 * child print.
 * SOLUTION=0 the bug: "My pid is ..." appears twice (the unflushed stdio
 * buffer is copied by fork)
 * SOLUTION=1 fflush(stdout) before fork()
 * SOLUTION=2 no stdio buffer for stdout: setvbuf(stdout, NULL, _IONBF, 0)
 * SOLUTION=3 the parent leaves with _exit(): its copy of the buffer is
 * never flushed,
 * the child's copy is printed together with the child's line
 * Make builds the four variants: fork_pid_0 ... fork_pid_3.
 */
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>

#include "utils.h"

#ifdef SOLUTION
#define SOLUTION 1
#endif
```

```
int main(void)
{
    #if SOLUTION == 2
        exit_if(setvbuf(stdout, NULL, _IONBF, 0) != 0, "setvbuf");
    #endif

    printf("My pid is %d / ", getpid());    /* no '\n': stays in the buffer
    */
    #if SOLUTION == 1
        fflush(stdout);
    #endif

    pid_t pid = fork();
    exit_if(pid == -1, "fork");

    if (pid == 0) {                          /* child */
        printf("child %d, parent %d\n", getpid(), getppid());
        exit(EXIT_SUCCESS);                  /* exit flushes the child's
    buffer */
    }

    /* Parent: prints nothing after fork, but must wait for its child. */
    int status;
    exit_if(waitpid(pid, &status, 0) == -1, "waitpid");
    #if SOLUTION == 3
        _exit(EXIT_SUCCESS);                  /* do not flush the parent's
    copy */
    #else
        return EXIT_SUCCESS;                  /* returning from main =
    exit(): flushes */
    #endif
}
```

Sortida real

```
$ ./fork_pid_0 | cat
My pid is 356446 / child 356448, parent 356446
My pid is 356446 /
```

Sortida real

```
$ ./fork_pid_1 | cat
My pid is 356458 / child 356460, parent 356458
```

Sortida real

```
$ ./fork_pid_3 | cat
My pid is 356482 / child 356484, parent 356482
```

El problema. «My pid is ... /» s'ha quedat a la memòria intermèdia de `stdout`, que viu a la memòria del procés. `fork` copia tota la memòria, memòria intermèdia inclosa: pare i fill tenen el mateix text pendent i tots dos el buiden en acabar, així que apareix dues vegades tot i que el pare «no imprimeix res». **Solucions:** (1) `fflush(stdout)` abans de `fork`; (2) treure la memòria intermèdia amb `setvbuf(..., _IONBF, 0)`; (3) fer que el pare acabi amb `_exit`, que no buida la seva còpia: el text surt una vegada, des del fill. La 1 és la més clara; la regla general és *buidar les memòries intermèdies abans de `fork`*.

TD3 · 4

● Result · provat

launch : redirigir i executar un altre programa

Enunciat. `./launch sortida ordre arg1 arg2...` s'ha de comportar com `ordre arg1 arg2... > sortida`: escriu la línia d'ordres a `sortida`, crea un procés fill que redirigeix la seva sortida estàndard amb `dup2` i executa l'ordre amb `execvp`, i el pare informa de com ha acabat (`wait(2)`). Prova'l amb un programa que provoqui una violació de segment.

Solució

```
C tds/td3/launch.c

/*
 * TD3 - exercise 4: ./launch output command [args...] behaves like command
 args... > output
 * 1. writes the command line into `output`
 * 2. forks; the child redirects its stdout to `output` with dup2 and runs
 the command (execvp)
 * 3. the parent waits and reports how the command ended (exit status or
 signal)
 */
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
```

```

#include <string.h>
#include <sys/wait.h>
#include <unistd.h>

#include "utils.h"

int main(int argc, char *argv[])
{
    if (argc < 3) {
        fprintf(stderr, "usage: %s output command [args...]\n", argv[0]);
        return EXIT_FAILURE;
    }

    /* O_TRUNC: like "> output", previous content is discarded. 0644 = rw-r--
r-- */
    int fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    exit_if(fd == -1, argv[1]);

    for (int i = 2; i < argc; i++) {
        exit_if(write_all(fd, argv[i], strlen(argv[i])) == -1, "write");
        exit_if(write_all(fd, i < argc - 1 ? " " : "\n", 1) == -1, "write");
    }

    pid_t pid = fork();
    exit_if(pid == -1, "fork");

    if (pid == 0) {
        /* child */
        exit_if(dup2(fd, STDOUT_FILENO) == -1, "dup2");
        exit_if(close(fd) == -1, "close"); /* 1 now points to the
file */
        execvp(argv[2], argv + 2); /* only returns on
error */
        perror(argv[2]);
        _exit(127); /* like the shell:
command not found */
    }

    exit_if(close(fd) == -1, "close"); /* the parent does not
need it */

    int status;
    exit_if(waitpid(pid, &status, 0) == -1, "waitpid");
    if (WIFEXITED(status)) {
        fprintf(stderr, "launch: %s exited with status %d\n", argv[2],
WEXITSTATUS(status));
    }
  
```

```

    return WEXITSTATUS(status);
  }
  if (WIFSIGNALED(status)) {
    fprintf(stderr, "launch: %s killed by signal %d (%s)%s\n", argv[2],
WTERMSIG(status),
           signame(WTERMSIG(status)), WCOREDUMP(status) ? ", core
dumped" : "");
    return 128 + WTERMSIG(status);           /* shell convention */
  }
  return EXIT_FAILURE;
}

```

C tds/td3/segfault.c

```

/*
 * TD3 - exercise 4: a program that dies with SIGSEGV, to test the error path
of launch.
 * Address 0 is never mapped: the MMU raises a page fault, the kernel finds
no mapping and
 * sends SIGSEGV to the process.
 */
#include <stdio.h>
#include <stdlib.h>

#include "utils.h"

static void gen_segfault(void)
{
    volatile int *p = NULL;    /* volatile: the compiler must really do the
store */
    *p = 42;
}

int main(void)
{
    printf("about to write at address 0...\n");
    fflush(stdout);
    gen_segfault();
    return EXIT_SUCCESS;      /* never reached */
}

```

Sortida real

```
$ ./launch out.txt ls -1 demo; cat out.txt
launch: ls exited with status 0
ls -1 demo
a.txt
b.txt
```

Sortida real

```
$ ./launch out.txt ./segfault
launch: ./segfault killed by signal 11 (SIGSEGV), core dumped
[exit status 139]
```

Sortida real

```
$ ./launch out.txt nocommand
nocommand: No such file or directory
launch: nocommand exited with status 127
[exit status 127]
```

- `execvp` substitueix el programa del procés *conservant-ne els descriptors*: per això la redirecció es fa al fill, entre `fork` i `exec`. Així funciona el shell.
- `execvp` només torna si falla: la línia següent sempre és un error (127 = «no trobat», convenció del shell). Es fa servir `_exit` perquè el fill no buidi les memòries intermèdies copiades del pare.
- `WIFEXITED` / `WEXITSTATUS` i `WIFSIGNALED` / `WTERMSIG` descodifiquen l'estat. $139 = 128 + 11$ (SIGSEGV), la mateixa convenció que fa servir el shell a `$?`.

⚠ Corregit

La versió heretada (`redirection.c`) obria la sortida sense `O_TRUNC` (en quedaven restes d'execucions anteriors més llargues) i ignorava el resultat de `write`; l'enunciat l'anomenava `lance`.

Exploració: la memòria virtual i la vida d'un procés

Enunciat. Quatre programes curts per *mesurar* el que expliquen les diapositives 56–82: (1) `zones` imprimeix l'adreça del codi, les dades, el heap, un `malloc` gran, `printf` i la pila, i la regió de `/proc/self/maps` que conté cadascuna; executa'l diverses vegades. (2) `lazy` reserva 1 GiB i compta la memòria resident i les fallades de pàgina abans i després de tocar-ne la meitat. (3) `cow` omple 256 MiB, fa fork i compta les fallades del fill en llegir i en escriure. (4) `lifecycle` mostra un zombi amb `ps` abans i després de `waitpid`, i un orfe adoptat.

Solució i sortides reals

C tds/td3/zones.c

```
/*
 * TD3 - exploration 1: the memory zones of a process (original slide 62,
 * updated).
 * Prints the address of the code, the data, the heap, a large malloc, a
 * library function and the stack,
 * and the region of /proc/self/maps that contains each one. Run it twice:
 * with ASLR (address space
 * layout randomization) the addresses change on every run, but the order of
 * the zones does not.
 */
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#include "utils.h"

int global_array[10];           /* uninitialized global: .bss */
static int initialized = 42;    /* initialized global: .data */

/* Finds the line of /proc/self/maps whose range contains addr and prints its
 * permissions and name. */
static void show(const char *what, const void *addr)
{
    FILE *maps = fopen("/proc/self/maps", "r");
    exit_if(maps == NULL, "/proc/self/maps");
```

```

char line[512];
uintptr_t a = (uintptr_t)addr;
while (fgets(line, sizeof line, maps) != NULL) {
    unsigned long start, end;
    char perms[5], path[256] = "";
    if (sscanf(line, "%lx-%lx %4s %*s %*s %*s %255[^\n]", &start, &end,
perms, path) < 3)
        continue;
    if (a ≥ start && a < end) {
        char *name = path;
        while (*name == ' ')
            name++;
        printf("%-24s %#14lx  %s  %s\n", what, (unsigned long)a, perms,
*name ? name : "(anonymous)");
        break;
    }
}
exit_if(fclose(maps) == EOF, "fclose");
}

int main(void)
{
    int local = 0;
    void *small = malloc(10);
    void *big = malloc(64 << 20);          /* above M_MMAP_THRESHOLD (128 KiB):
glibc uses mmap */
    exit_if(small == NULL || big == NULL, "malloc");

    printf("%-24s %14s  %s  %s\n", "zone", "address", "perm", "region");
    show("code   (main)", (const void *)main);
    show("data   (initialized)", &initialized);
    show("bss    (global_array)", global_array);
    show("heap   (malloc 10 B)", small);
    show("heap   (end: sbrk(0))", (char *)sbrk(0) - 1);
    show("mmap   (malloc 64 MiB)", big);
    show("libc   (printf)", (const void *)printf);
    show("stack  (local)", &local);

    free(big);
    free(small);
    return EXIT_SUCCESS;
}

```

Sortida real

```
$ ./zones
zone                address  perm  region
code  (main)        0x5f935f9d1350  r-xp  ~/fso/tds/bin/td3/zones
data  (initialized)  0x5f935f9d5010  rw-p  ~/fso/tds/bin/td3/zones
bss   (global_array) 0x5f935f9d5040  rw-p  ~/fso/tds/bin/td3/zones
heap  (malloc 10 B)    0x5f938dc96010  rw-p  [heap]
heap  (end: sbrk(0)) 0x5f938dcb6fff  rw-p  [heap]
mmap  (malloc 64 MiB) 0x71b340fff010  rw-p  (anonymous)
libc  (printf)       0x71b345064080  r-xp  /usr/lib/x86_64-linux-
gnu/libc.so.6
stack (local)       0x7fff6c1e68b4  rw-p  [stack]
```

Sortida real

```
$ for i in 1 2 3; do ./zones | grep main; done
code  (main)        0x57a1f784a350  r-xp  ~/fso/tds/bin/td3/zones
code  (main)        0x5d648e064350  r-xp  ~/fso/tds/bin/td3/zones
code  (main)        0x56e46bc21350  r-xp  ~/fso/tds/bin/td3/zones
```

L'ordre de les zones es manté (codi, dades i heap a baix; biblioteques i `mmap` a sobre; la pila a dalt de tot), però la base canvia a cada execució: **ASLR**. El `malloc` de 64 MiB no és al heap: glibc fa servir `mmap` per sobre de 128 KiB.

C tds/td3/lazy.c

```
/*
 * TD3 - exploration 2: lazy allocation (original slides 59 and 61).
 * malloc of a large block returns at once and costs almost no physical
 * memory: the kernel only
 * reserves virtual addresses. Each page gets a physical frame the first time
 * it is touched, through a
 * (minor) page fault. The program measures the resident memory (VmRSS) and
 * the page faults
 * (getrusage) before and after touching half of the block.
 * ./lazy [MiB]          default: 1024 MiB
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/resource.h>
#include <time.h>
#include <unistd.h>
```

```

#include "utils.h"

static long minor_faults(void)
{
    struct rusage ru;
    exit_if(getrusage(RUSAGE_SELF, &ru) == -1, "getrusage");
    return ru.ru_minflt;
}

/* Resident set size in KiB, from /proc/self/status. */
static long rss_kib(void)
{
    FILE *f = fopen("/proc/self/status", "r");
    exit_if(f == NULL, "/proc/self/status");
    char line[256];
    long kib = -1;
    while (fgets(line, sizeof line, f) != NULL)
        if (sscanf(line, "VmRSS: %ld kB", &kib) == 1)
            break;
    exit_if(fclose(f) == EOF, "fclose");
    return kib;
}

static double now(void)
{
    struct timespec ts;
    exit_if(clock_gettime(CLOCK_MONOTONIC, &ts) == -1, "clock_gettime");
    return ts.tv_sec + ts.tv_nsec / 1e9;
}

static void report(const char *step, long f0)
{
    printf("%-34s VmRSS = %8ld KiB    page faults so far = %ld\n", step,
    rss_kib(), minor_faults() - f0);
}

int main(int argc, char *argv[])
{
    size_t mib = argc > 1 ? (size_t)atol(argv[1]) : 1024;
    size_t size = mib << 20, page = (size_t)sysconf(_SC_PAGESIZE);
    long f0 = minor_faults();

    report("start", f0);
    char *p = malloc(size);
    exit_if(p == NULL, "malloc");

```

```
printf("malloc(%zu MiB) returned %p\n", mib, (void *)p);
report("after malloc", f0);

long before = minor_faults();
double t0 = now();
for (size_t i = 0; i < size / 2; i += page)    /* touch one byte of
every page of the first half */
    p[i] = 1;
double t1 = now();
long faults = minor_faults() - before;
report("after touching half of the pages", f0);
printf("pages touched: %zu (%zu bytes each)    page faults: %ld    %.0f ns
per fault\n",
        size / 2 / page, page, faults, faults ? (t1 - t0) * 1e9 / faults :
0.0);

free(p);
report("after free", f0);
return EXIT_SUCCESS;
}
```

Sortida real

```
$ ./lazy 1024
start                                VmRSS =      1756 KiB    page faults so far
= 0
malloc(1024 MiB) returned 0x7fadb67ff010
after malloc                          VmRSS =      1952 KiB    page faults so far
= 8
after touching half of the pages    VmRSS =   526300 KiB    page faults so far
= 131081
pages touched: 131072 (4096 bytes each)    page faults: 131073    1968 ns per
fault
after free                            VmRSS =      2012 KiB    page faults so far
= 131081
```

`malloc(1 GiB)` no canvia la memòria resident: només reserva adreces. Cada pàgina tocada per primera vegada provoca una fallada de pàgina en què el nucli assigna i posa a zero un marc: tantes fallades com pàgines.

C tds/td3/cow.c

```
/*
 * TD3 - exploration 3: copy-on-write after fork (original slide 74).
 * The parent fills a block, so all its pages are resident. After fork,
```

```

parent and child share the
    * physical pages, marked read-only. The child reads the whole block (no
copy: no page faults) and then
    * writes to a quarter of the pages: each first write causes a page fault in
which the kernel copies
    * that single page. The page faults are counted with getrusage.
    * ./cow [MiB]          default: 256 MiB
    */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/resource.h>
#include <sys/wait.h>
#include <time.h>
#include <unistd.h>

#include "utils.h"

static long minor_faults(void)
{
    struct rusage ru;
    exit_if(getrusage(RUSAGE_SELF, &ru) == -1, "getrusage");
    return ru.ru_minflt;
}

static double now(void)
{
    struct timespec ts;
    exit_if(clock_gettime(CLOCK_MONOTONIC, &ts) == -1, "clock_gettime");
    return ts.tv_sec + ts.tv_nsec / 1e9;
}

int main(int argc, char *argv[])
{
    size_t mib = argc > 1 ? (size_t)atol(argv[1]) : 256;
    size_t size = mib << 20, page = (size_t)sysconf(_SC_PAGESIZE), pages =
size / page;
    char *p = malloc(size);
    exit_if(p == NULL, "malloc");
    memset(p, 'x', size);                                /* every page is
now resident */
    printf("parent: %zu MiB filled (%zu pages of %zu bytes)\n", mib, pages,
page);
    fflush(stdout);                                       /* do not
duplicate the stdio buffer */

```

```
double t0 = now();
pid_t pid = fork();
exit_if(pid == -1, "fork");
if (pid == 0) {
    printf("fork took %.2f ms (only the page tables are copied)\n",
(now() - t0) * 1e3);

    long f = minor_faults();
    unsigned long sum = 0;
    for (size_t i = 0; i < size; i += page)
        sum += (unsigned char)p[i];
    printf("child reads %zu pages: %ld page faults (shared, nothing
copied; sum %lu)\n", pages, minor_faults() - f, sum);

    f = minor_faults();
    double t1 = now();
    for (size_t i = 0; i < size / 4; i += page)
        p[i] = 'y';
    double t2 = now();
    long faults = minor_faults() - f;
    printf("child writes %zu pages: %ld page faults (one copy per page),
%.0f ns per copy\n",
        pages / 4, faults, faults ? (t2 - t1) * 1e9 / faults : 0.0);
    free(p);
    exit(EXIT_SUCCESS);
}

int status;
exit_if(waitpid(pid, &status, 0) == -1, "waitpid");
printf("parent: the child exited with status %d; my copy still says
'%c'\n", WEXITSTATUS(status), p[0]);
free(p);
return EXIT_SUCCESS;
}
```

Sortida real

```
$ ./cow 256
parent: 256 MiB filled (65536 pages of 4096 bytes)
fork took 6.70 ms (only the page tables are copied)
child reads 65536 pages: 0 page faults (shared, nothing copied; sum 7864320)
child writes 16384 pages: 16384 page faults (one copy per page), 2753 ns per
copy
parent: the child exited with status 0; my copy still says 'x'
```

Després de `fork`, llegir no copia res (0 fallades); escriure provoca exactament una fallada —una còpia— per pàgina escrita. Per això fer `fork` d'un procés de 256 MiB triga mil·lisegons: només copia les taules de pàgines.

```
C tds/td3/lifecycle.c

/*
 * TD3 - exploration 4: the end of a process: zombies and orphans (original
 * slides 80-82 and 88).
 * 1. A child that exits stays as a ZOMBIE (state Z, <defunct>) until its
 * parent calls wait: the
 *     kernel keeps its PID and exit status. ps shows it before and after
 * waitpid.
 * 2. A process whose parent dies is an ORPHAN: the kernel gives it a new
 * parent, init (PID 1) or the
 *     nearest ancestor marked as "subreaper" (on a desktop, usually systemd
 * --user).
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>
#include <time.h>
#include <unistd.h>

#include "utils.h"

static void pause_ms(long ms)
{
    struct timespec ts = { ms / 1000, (ms % 1000) * 1000000L };
    while (nanosleep(&ts, &ts) == -1)
        ;
}

static void ps(pid_t pid)
{
    char cmd[128];
    snprintf(cmd, sizeof cmd, "ps -o pid,ppid,stat,comm -p %d | sed 's/^/'", (int)pid);
    fflush(stdout);
    if (system(cmd) == -1)
        perror("system");
}

/* Name of a process, from /proc/PID/comm. */
static void comm_of(pid_t pid, char *name, size_t len)
```

```

{
    char path[64];
    snprintf(path, sizeof path, "/proc/%d/comm", (int)pid);
    FILE *f = fopen(path, "r");
    if (f == NULL || fgets(name, (int)len, f) == NULL)
        snprintf(name, len, "?");
    name[strcspn(name, "\n")] = '\0';
    if (f != NULL)
        fclose(f);
}

int main(void)
{
    /* 1. Zombie */
    pid_t child = fork();
    exit_if(child == -1, "fork");
    if (child == 0)
        _exit(7);
    pause_ms(200);
    printf("1) the child %d has exited, but nobody has called wait yet:\n",
(int)child);
    ps(child);
    int status;
    exit_if(waitpid(child, &status, 0) == -1, "waitpid");
    printf("    waitpid → WIFEXITED = %d, WEXITSTATUS = %d; now ps finds
nothing:\n", WIFEXITED(status), WEXITSTATUS(status));
    ps(child);

    /* 2. Orphan: the middle process exits, the grandchild is adopted */
    printf("2) orphan: my PID is %d\n", (int)getpid());
    fflush(stdout);
    pid_t middle = fork();
    exit_if(middle == -1, "fork");
    if (middle == 0) {
        pid_t grandchild = fork();
        exit_if(grandchild == -1, "fork");
        if (grandchild == 0) {
            pid_t before = getppid();
            pause_ms(200);
            /* meanwhile its parent
exits */
            pid_t after = getppid();
            char name[64];
            comm_of(after, name, sizeof name);
            printf("    grandchild %d: parent was %d, now it is %d (%s)\n",
(int)getpid(), (int)before, (int)after, name);

```

```
        exit(EXIT_SUCCESS);
    }
    exit(EXIT_SUCCESS);                /* the grandchild becomes
an orphan */
    }
    exit_if(waitpid(middle, NULL, 0) == -1, "waitpid");
    pause_ms(400);                    /* let the grandchild
report */
    return EXIT_SUCCESS;
}
```

Sortida real

```
$ ./lifecycle
1) the child 356667 has exited, but nobody has called wait yet:
    PID    PPID STAT COMMAND
    356667 356666 Z    lifecycle
waitpid → WIFEXITED = 1, WEXITSTATUS = 7; now ps finds nothing:
    PID    PPID STAT COMMAND
2) orphan: my PID is 356666
grandchild 356679: parent was 9771, now it is 9771 (systemd)
```

El fill acabat mostra l'estat `Z` fins al `waitpid`, que en recull el codi (7). El nét orfe no va a `init` sinó al `systemd` de la sessió d'usuari, un *subreaper* (`prctl(PR_SET_CHILD_SUBREAPER)`).

Mira-ho animat → A4.1 memòria virtual · A4.2 còpia en escriptura · A4.3 estats

Llista de comprovació

Repte amb IA generativa · Un mini shell



Objectiu. Converteix `launch` en un intèrpret d'ordres mínim: llegeix línies, executa ordres amb `fork`/`execvp`, admet `>` fitxer i mostra l'estat de sortida.

Prompt inicial. Fes-lo servir a l'eina que prefereixis (Claude, ChatGPT, Gemini, Copilot, Codex...) amb la versió *resolta* del laboratori com a context. Demana primer un pla i revisa cada pas abans d'acceptar-lo.

Prompt

A partir del meu `launch.c` [enganxa'l], ajuda'm a escriure un mini shell en C que: llegeixi línies amb `getline`, les divideixi en paraules, executi l'ordre amb `fork` + `execvp`, admeti la redirecció `>` fitxer amb `dup2`, implementi `cd` i `exit` com a ordres internes (explica per què `cd` no pot ser un programa extern) i imprimeixi `[status N]` o `[signal SIGxxx]` quan acabi una ordre. Demana'm que escrigui cada part abans d'ensenyar-me la teva.

Com comprovar el resultat

- `ls -l > out.txt` crea el fitxer i el shell continua funcionant.
- `cd /tmp` seguit de `pwd` imprimeix `/tmp`.
- Executar `./segfault` mostra el senyal i no deixa cap procés zombi (`ps`).

Declara l'ús de la IA: eina, prompts rellevants, què has comprovat tu i què has corregit. El que compta és que puguis explicar i verificar cada decisió.